

INHA UNIVERSITY TASHKENT

DEPARTMENT OF CSE & ICE

FALL SEMESTER 2024

SOC 3010 - OPERATING SYSTEMS

HOME ASSIGNMENT 2

INSTRUCTIONS :
- ALL HOME ASSIGNMENTS ARE TO BE COMPLETED IN GROUPS OF 4 TO 6 MEMBERS
- SCREEN SHOTS ARE TO BE PROVIDED FOR ALL THE QUESTIONS
- SHARING OF THE QUESTIONS AND SCREENSHOTS IS NOT ALLOWED
- HOME ASSIGNMENT REPORT SHOULD BE PREPARED USING THE TEMPLATE PROVIDED AT THE ECLASS
- A HARDCOPY OF THE TEAM LEADER'S REPORT (CONTAINING ONLY TEAM LEADER'S PART OF THE WORK) MUST BE HANDED IN BY THE TEAM LEADER BY VISITING MY OFFICE ON MONDAY 14TH OCTOBER AFTERNOON ANYTIME BETWEEN 2PM & 5PM.
- EVERY MEMBER OF THE TEAM MUST UPLOAD THE SOFTCOPY OF THE REPORT AT THE IUT ECLASS PORTAL
- LAST DATE FOR SUBMISSION OF THE HOME ASSIGNMENT2 REPORT AT THE ECLASS PORTAL IS SUNDAY 13TH OCTOBER 2024
- LATE SUBMISSIONS ARE NOT ENTERTAINED, ADHERE TO THE DEADLINE STRICTLY
- READ THE QUESTIONS CORRECTLY & CAREFULLY

QUESTIONS :

PART I : PROCESS CREATION & TERMINATION

EVERY MEMBER OF THE TEAM MUST EXECUTE ALL THE PROGRAMS GIVEN HERE IN PART I QUESTIONS 1, 2 & 3 ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS.

1. You are given the following set of programs to understand clearly the child process creation and termination phases.

You are required to go through each of these programs and do the following:

- a) Describe in detail the sequence of operations carried out by each of the parent and child processes created and sequence of their terminations.**
- b) Run each program separately and provide all the outputs of the parent and child processes and provide your analysis on these outputs.**
- c) Provide screen shots of all the results including the program.**

(i) Program Name : forkchild1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>
void unix_error(char *msg)
{
    fprintf(stderr, "%s : %s\n", msg, strerror(errno));
    exit(0);
}
pid_t Fork(void)
{
    pid_t pid;
    if(( pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}
int main()
{
    pid_t pid;
    int val=1;
```

```

printf("start of parent process : val=%d, pid=%d \n", val, getpid());
if ((pid = Fork()) == 0)
{
    printf("created process child : val=%d, pid=%d \n", ++val, getpid());
    exit(0);
}
printf("now in parent : val=%d, pid=%d \n", --val, getpid());
exit(0);
}

```

(ii) Program Name : forkchild2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;

    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

int main()
{
    printf("L0\n");
    Fork();
    printf("L1 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
    Fork();
    printf("L2 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
    Fork();
    printf("L3 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
    Fork();
    printf("Bye : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
}

```

(iii) Program Name : forkchild3.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */

```

```

{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}
pid_t Fork(void)
{
    pid_t pid;
    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

int main()
{
    printf("L0 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
    if (Fork() == 0)
    {
        printf("L1 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
        if (Fork() == 0)
        {
            printf("L2 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
            if (Fork() == 0)
            {
                printf("L3 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
            }
            printf("L4 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
        }
        printf("L5 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
    }
    printf("Bye\n");
}

```

(iv) Program Name : forkchild4.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}
pid_t Fork(void)
{
    pid_t pid;

    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

```

```

int main()
{
    pid_t pid;
    printf("hello F0 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
    if (Fork() == 0 )
    {
        printf("hello F1 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
        if (Fork() != 0)
        {
            printf("hello F2 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
            if (Fork() == 0)
            {
                printf("hello F3 : CHILD PID = %d, PARENT PID = %d\n", getpid(),
getppid());
            }
            printf("hello F4 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
        }
        printf("hello F5 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
        if (Fork() == 0)
        {
            printf("hello F6 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
            if (Fork() != 0)
            {
                printf("hello F7 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
            }
            printf("hello F8 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
        }
        printf("hello F9 : CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
    }
    printf("BYE F10: CHILD PID = %d, PARENT PID = %d\n", getpid(), getppid());
}

```

(v) Program Name : forkchild5.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;
    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

```

```

int main()
{
    pid_t pid;
    printf("HELLO!!!");
    if (Fork() == 0 )
    {
        printf(" OPERATING SYSTEMS ");
        if (Fork() != 0)
        {
            printf("HOME ASSIGNMENT 2 ");
            if (Fork() == 0)
                printf(" SUBMISSION ");
            printf(" DEADLINE ");
        }
        printf(" AFTER 2 WEEKS");
        if (Fork() != 0)
        {
            printf("SUNDAY 13TH OCTOBER 2024 --- ");
            if (Fork() == 0)
                printf(" GOOD LUCK ");
        }
    }
    printf("BYE \n");
}

```

2. You are given the following programs to show the creation of orphan/Zombie processes :

You are required to go through each of these programs and do the following:

- a) Run each program separately and provide all the outputs of the parent and child processes and provide your analysis on these outputs.**
- b) Provide screen shots of all the results including the program.**
- c) Identify the child processes in the Zombie states by running the ps -la command and provide the screenshots with proper explanation**
- d) Also use appropriate commands to kill all the Zombie processes and provide necessary screen shots to show that these processes no longer exist.**

(i) Program Name : orphachild.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>

```

```

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;
    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

int main()
{
    if (Fork() == 0)
    {
        /* Child */
        printf("RUNNING CHILD - CHILD PID = %d\n", getpid());
        printf("HELLO - I AM GOING TO SOON BECOME AN ORPHAN CHILD AS MY PARENT WILL GET TERMINATED SOON \n");
        printf("OPEN ANOTHER TERMINAL AND SEE ME RUNNING WITHOUT THE PARENT BY EXECUTING THE COMMAND ps -la\n");
        printf("TO KILL ME USE THE COMMAND : kill -9 mypid \n");
        while (1)
            ; /* Infinite loop */
    }
    else
    {
        sleep(10);
        printf("PARENT NOW TERMINATING - PARENT PID = %d\n", getpid());
        exit(0);
    }
}

```

(ii) Program Name : zombiechild1.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;
    if ((pid = fork()) < 0)

```

```

        unix_error("Fork error");
    return pid;
}

int main()
{
    pid_t pid[100];
    int i, n;
    printf("Number of Child processes n=");
    scanf("%d", &n);
    printf("HELLO FROM PARENT, PARENT PID = %d\n", getpid());
    for(i=0; i < n; i++)
    {
        printf("PARENT : NOW I AM CREATING CHILD NO-%d \n",i);
        if (Fork() == 0 )
        {
            pid[i]= getpid();

            printf("*****\n");
            printf("* HELLO FROM CHILD , My pid is %d, I HAVE BEEN PAUSED\n", pid[i]);
            printf("* OPEN ANOTHER TERMINAL KILL ME BY GETTING MY PID using ps -l AND THEN EXECUTE THE COMMAND kill -9 mypid\n");
            printf("* NOW I HAVE BECOME A ZOMBIE(DEFUNCT) PROCESS - CHECK MY STATUS BY USING THE COMMAND ps -la\n");
            printf("* IF YOU WANT TO KILL ALL ZOMBIE PROCESSES THEN KILL THE PARENT PROCESS BY USING kill -9 parentpid AND CHECK USING ps -la\n");
            printf("*****\n");
            pause();
            exit(0);/*CHILD*/
        }
    }
    printf("PARENT : I AM STILL RUNNING Running - PARENT PID = %d\n", getpid());
    while (1)
        ; /* Infinite loop */
}

```

(iii) Program Name : zombiechild2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>
void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}
pid_t Fork(void)

```

```

{
    pid_t pid;

    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

int main()
{
    pid_t pid[100];
    int i, n, ttime;
    printf("Number of Child processes n=");
    scanf("%d", &n);
    printf("HELLO FROM PARENT, pid = %d\n", getpid());
    for(i=0; i < n; i++)
        if (Fork() == 0 )
        {
            pid[i]= getpid();
            ttime = 5 *(i+1);
            printf("HELLO FROM CHILD , My pid is %d, I am going to SLEEP for %d secs\n",pid[i],
ttime);
            sleep(ttime);
            printf("CHILD [%d] : I just WOKE UP -- EXITING BYE BYE\n", pid[i]);
            exit(0);/*CHILD*/
        }
    printf("I AM STILL RUNNING - Running Parent, PID = %d\n", getpid());
    while (1)
        ; /* Infinite loop */
}

```

3. You are given the following programs to show how parent process reaps child processes thereby avoiding creation of orphan or zombie processes :

You are required to go through each of these programs and do the following:

- a) Run each program separately and provide all the outputs of the parent and child processes and provide your analysis on these outputs.**
- b) Provide screen shots of all the results including the program.**
- c) Also comment on the order in which child processes are reaped by the parent process.**

(i) Program Name : reapchildnorder.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>

```

```

#include <unistd.h>
#include <errno.h>
#include <time.h>
#include <sys/wait.h>
#define N 20

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;

    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

int main()
{
    pid_t pid[N], wpid;
    int i, child_status;
    for (i = 0; i < N; i++)
        if ((pid[i] = Fork()) == 0)
        {

            printf("CHILD[%d] IS GOING TO SLEEP FOR 50 SECS \n", getpid());
            printf("CHILD[%d] : TIME BEFORE EXIT IN CLK CYCLES = %d\n", getpid(), clock());
            sleep(50);
            exit(100+i); /* Child */
        }
    for (i = 0; i < N; i++)
    {
        /* Parent */
        printf("PARENT REAPING CHILD NOT IN ANY SPECIFIC ORDER \n");
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
        {
            printf("PARENT : TIME AFTER WAIT FOR TERMINATION OF CHILD[%d] in CLK
cycles = %d\n", wpid, clock());
            printf("Child  %d  TERMINATED  WITH  EXIT  STATUS  %d\n",  wpid,
WEXITSTATUS(child_status));
        }
        else
            printf("Child %d TERMINATE ABNORMALLY\n", wpid);
    }
}

```

(ii) Program Name : reapchildorder.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <errno.h>
void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;
    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

int main()
{
    int status, i, n, ttime;
    pid_t pid[100], retpid;
    printf("Number of Child processes n=");
    scanf("%d", &n);
    printf("HELLO FROM PARENT, pid = %d\n", getpid());
    /* Parent creates N children */
    for (i = 0; i < n; i++)
        if ((pid[i]=Fork()) == 0) /* Child */
        {
            ttime = 5 * (i+1);
            printf("HELLO FROM CHILD , My PID is %d, I am going to SLEEP for %d secs \n",
getpid(),ttime);
            sleep(ttime);
            printf("CHILD %d : I JUST WOKE UP - I AM EXITING - BYE BYE \n", getpid());
            exit(100+i); /*CHILD*/
        }
    /* Parent reaps N children in particular order */
    printf("PARENT REAPING CHILD IN PARTICULAR ORDER \n");
    i = 0;
    while ((retpid = waitpid(pid[i++], &status, 0)) > 0)
    {
        if (WIFEXITED(status))
        {
            printf("PARENT %d : I AM NOW REAPING CHILD %d \n", getpid(), retpid);
            printf("CHILD %d terminated with exit status %d FROM PARENT pid = %d\n", retpid,
WEXITSTATUS(status), getpid());
        }
        else
    }
```

```

        printf("child %d terminated abnormally\n", retpid);
    }
    /* The only normal termination is if there are no more children */
    if (errno != ECHILD)
        unix_error("waitpid error");
    exit(0);
}

```

(iii) Program Name : reapchildrorder.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <errno.h>

void unix_error(char *msg) /* Unix-style error */
{
    fprintf(stderr, "%s: %s\n", msg, strerror(errno));
    exit(0);
}

pid_t Fork(void)
{
    pid_t pid;
    if ((pid = fork()) < 0)
        unix_error("Fork error");
    return pid;
}

int main()
{
    int status, i, n;
    pid_t pid[100], retpid;
    printf("Number of Child processes n=");
    scanf("%d", &n);
    printf("HELLO FROM PARENT, PARENT PID = %d\n", getpid());
    /* Parent creates N children */
    for (i = 0; i < n; i++)
        if ((pid[i] = Fork()) == 0) /* Child */
        {
            printf("HELLO FROM CHILD , CHILD[%d] PID = %d\n", i, getpid());
            sleep(1*(i+1));
            exit(100+i); /*CHILD*/
        }
    /* Parent reaps N children in particular order */
    printf("PARENT REAPING CHILD IN REVERSE ORDER \n");
    i = n-1;
    while ((retpid = waitpid(pid[i--], &status, 0)) > 0)
    {
        if (WIFEXITED(status))

```

```

        printf("CHILD %d terminated with exit status %d , Its PARENT PID = %d\n", retpid,
WEXITSTATUS(status), getpid());
    else
        printf("child %d terminated abnormally\n", retpid);
}
/* The only normal termination is if there are no more children */
if (errno != ECHILD)
    unix_error("waitpid error");
exit(0);
}

```

PART II : INTERPROCESS COMMUNICATION

EVERY MEMBER OF THE TEAM MUST EXECUTE ALL THE PROGRAMS GIVEN HERE IN PART II QUESTIONS 1 & 2 ON HIS/HER LAPTOP AND INCLUDE THE SCREENSHOTS OF THESE IN THEIR REPORTS.

THREAD SYNCHRONIZATION :

1. You are given the following program showing how variables are shared in threads.
 - a) Run this program and provide the outputs produced by each thread.
 - b) Provide screen shots of all the results including the program.
 - c) Also identify clearly the variables shared and not shared between the threads.

Program Name : threadvshare.c

```

/* Program illustrating sharing of variables in Threadds */
/* Compile the program using the command on linux : $ gcc -pthread -o threadvshare
threadvshare.c */

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

```

```

#include <errno.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *),
void *arg);
void *thread(void *vargp);
char **ptr;    /* global var */

int main()
{
    long i;
    pthread_t  tid[2];
    char *msgs[2] = { "Hello from foo", "Hello from bar" };
    ptr = msgs;
    for (i = 0; i < 2; i++)
        pthread_create(&tid[i], NULL, thread, (void *)i);
    pthread_exit(NULL);
}

void *thread(void *vargp)
{
    long myid = (long)vargp;
    static int cnt = 0;
    printf("[%ld]:  %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
    return NULL;
}

```

2. You are given the following program to illustrate the effect of improper synchronization in threads.

Program name : badcntn.c

```

/*Program illustrating improper synchronization in Threaads */
/* Compile the program using the command on linux : $ gcc -pthread -o badcntn badcntn.c */
/* To execute the program enter: $ ./badcntn 100000 300 */

```

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <signal.h>
#include <sys/wait.h>
#include <unistd.h>
#include <errno.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *),
void *arg);
void *thread(void *vargp);
/* Global shared variable */
volatile long cnt = 0; /* Counter */

```

```

int main(int argc, char **argv)
{
    long niters;
    int i, nthreads;
    pthread_t tid[1000];
    niters = atoi(argv[1]);
    nthreads = atoi(argv[2]);
    for (i=0; i < nthreads; i++)
        pthread_create(&tid[i], NULL, thread, &niters);
    for (i=0; i < nthreads; i++)
        pthread_join(tid[i], NULL);
    /* Check result */
    if (cnt != (nthreads * niters))
        printf("BOOM! cnt=%ld\n", cnt);
    else
        printf("OK cnt=%ld\n", cnt);
    printf("PRESS CTRL-C to terminate or CTRL-Z to suspend \n");
    pause();
    exit(0);
}

/* Thread routine */
void *thread(void *vargp)
{
    long i, niters = *((long *)vargp);
    for (i = 0; i < niters; i++)
        cnt++;
    return NULL;
}

```

- a) **Run this program several times with the same inputs (number of iterations and number of threads specified as command line arguments For example - \$badcntn 100000 300) and identify the thread synchronization problem (shared variable involved) for getting different answers at different runs. Provide screen shots of all the results including the program.**
- b) **Modify the above program (call it goodcntn.c) using semaphores to provide proper synchronization (use semaphores P(&mutex) and V(&mutex) to control access to cnt shared variable) so that same results will be obtained for different runs of the program with the same input. Provide screen shots of all the results including the program.**

PART III : IDENTIFYING KERNEL SOURCE CODE

Identify the following programs, functions, macros, header files (discussed in your text book chapters) in the Kernel source code installed on your Linux system.

The book discusses older version of Linux 2.6, but you might be having a latest version of the Linux 4.4 or above installed on your system. Hence you may not find some of these, but you can always find most of them.

In order to do that, you need to do an exhaustive search in the Linux source directories (using linux commands like find, grep, etc.) to find out where these functions, macros and header files are located and defined and in which all files/directories these are referenced.

Then provide a table containing the list of functions/macros/header files, their purpose, where they are defined (name of directories and files), and lastly where they are referenced (list of directories & files). This can be used as a quick reference to access these functions, macros, header files in the source code directories. Also indicate (provide a list) of functions and macros which cannot be located in the source code directories.

CHAPTER 2 - MEMORY ADDRESSING

Page nos. to go through 59-77

Section - The Linear Address Fields -The following macros simplify page table handling- Page Table Handling Macros - PAGE_SHIFT, PAGE_SIZE, PAGE_MASK, PMD_SHIFT, PMD_SIZE, PMD_MASK, PUD_SHIFT, PUD_SIZE, PUD_MASK, PGDIR_SHIFT, PGDIR_SIZE, PGDIR_MASK, PTRS_PER_PTE, PTRS_PER_PMD, PTRS_PER_PUD, and PTRS_PER_PGD

SectionPage Table Handling - The kernel also provides several macros and functions to read or modify page table entries: pte_none(), pmd_none(), pud_none(), and pgd_none(), pte_present(), pmd_present(), pud_present(), and pgd_present(), pte_clear(), pmd_clear(), pud_clear(), and pgd_clear(), No pte_bad()

Several functions are offered to query the current value of any of the flags included in a Page Table entry: pte_read(), pte_write(), pte_exec(), pte_mkyoung(), pte_mkold(), pte_modify(p,v), set_pte

The macros that combine a page address and a group of protection flags into a 32bit page entry or perform the reverse operation of extracting the page address from a page table entry are: mk_pte(), mk_pte_phys, pte_quicklist, pgd_quicklist, pgt_cache_water array, check_pgt_cache(), check_pgt_cache()

Last round of functions and macros: `pgd_alloc( )`, `get_pgd_fast( )`, `get_pgd_slow( )`, `pmd_alloc(p,a)`, `pte_alloc(p,a)`, `get_pte_fast( )`, `get_pte_slow( )`, `pte_alloc_kernel( )`, `get_pte_kernel_slow()`, `get_pte_slow()`, `get_pte_kernel_slow()`, `pte_free()`, `pte_free_kernel()`, and `pgd_free()`, `pmd_free()` and `pmd_free_kernel()`, `free_one_pmd()`, `pte_free()`, `free_one_pgd()`, `free_one_pmd()`, `pmd_free()`. `SET_PAGE_DIR`, `new_page_tables()`, `clear_page_tables()`, `free_one_pgd()`, `free_page_tables()`, `clear_page_tables()`.

CHAPTER 4 – INTERRUPT HANDLING

Page nos. to go through 145-185

The following functions are used to insert gates in the IDT: `set_intr_gate(n,addr)`, `set_system_gate(n,addr)`, `set_system_intr_gate(n,addr)`, `set_trap_gate(n,addr)`, `set_task_gate(n,gdt)`, IDT initialization - `setup_idt()`, `ignore_int()`
Exception handling – `trap_init()`, `set_trap_gate`, `set_intr_gate`, `set_system_gate`, `set_system_intr_gate()` and `set_task_gate()` functions:

```
set_trap_gate(0,&divide_error);
set_trap_gate(1,&debug);set_intr_gate(2,&nmi);set_system_intr_gate(3,&int3);
set_system_gate(4,&overflow);set_system_gate(5,&bounds);
set_trap_gate(6,&invalid_op);set_trap_gate(7,&device_not_available);
set_task_gate(8,&double_fault);set_trap_gate(9,&coprocessor_segment_overrun);
set_trap_gate(10,&invalid_TSS);set_trap_gate(11,&segment_not_present);
set_trap_gate(12,&stack_segment);set_trap_gate(13,&general_protection);
set_intr_gate(14,&page_fault);set_trap_gate(16,&coprocessor_error);
set_trap_gate(17,&alignment_check);set_trap_gate(18,&machine_check);
set_trap_gate(19,&simd_coprocessor_error);set_system_gate(0x80,&system_call);
do_trap(), ret_from_exception(), die_if_kernel() or die_if_no_fixup(), do_IRQ, ret_from_intr(
), pci_read_config_byte() and pci_write_config_byte(), disable_irq(), enable_irq(), init_IRQ(),
setup_idt(), IRQn_interrupt(), setup_IO_APIC_irqs(), setup_local_APIC(), kirkd,
/proc/irq/n/smp_affinity, do_irq_balance(), set_ioapic_affinity_irq(), BUILD_IRQ,
BUILD_COMMON_IRQ, SAVE_ALL, do_8259A_IRQ(), mask_and_ack_8259A(),
handle_IRQ_event(), request_irq(), setup_x86_irq(), free_irq(), ret_from_sys_call(),
CALL_FUNCTION_VECTOR (vector 0xfb), RESCHEDULE_VECTOR (vector 0xfc),
INVALIDATE_TLB_VECTOR (vector 0xfd), send_IPI_all(), send_IPI_allbutself(),
send_IPI_self(), send_IPI_mask().
```

PART IV : LINUX KERNEL SOURCE CODE COMPILATION

- a) Compile and Build Linux Kernel from the source code installed.
- b) Describe clearly all the steps in the compilation and building of Linux Kernel from the kernel source files providing all the required steps and indicating Linux commands used.
- c) Test the new kernel compiled by rebooting with the new built kernel.
